Net: A Latency-First Encrypted Mesh Runtime for Heterogeneous Edge Computing

Dr. Laszlo Attila Vekony

We present Net, a latency-first encrypted mesh runtime for coordination on commodity hardware without controlling the physical layer. Its central abstraction is a non-localized event bus: unlike broker-based systems (Kafka) or single-process ring buffers (LMAX Disruptor) where the bus has a fixed location, Net's bus is not *at* a location—it *is* the mesh. Producers and consumers interact through a single event abstraction regardless of physical location, with routing, encryption, capability discovery, and failure recovery handled transparently by the runtime. Net inverts the assumptions of best-effort networking: nodes drop what they cannot process within a time window, derive trust from observed behavior rather than assuming it, and propagate state rather than maintaining connections. The system composes techniques from event sourcing, process migration, distributed causal ordering, capability-based scheduling, and self-healing mesh networking into a single runtime operating at nanosecond scheduling overhead. We describe a 9-layer architecture spanning encrypted UDP transport, cryptographic identity, channel-based authorization, hierarchical subnets, causal distributed state, migratable compute, extensible subprotocols, observational continuity, and contested-environment resilience. Micro-benchmarks on Apple M1 Max and Intel i9-14900K demonstrate 1.2–2.2ns header serialization (456–829M ops/sec), 34–52ns per-hop forwarding, ~20ns fast-path authorization checks, sub-300ns full failure-recovery cycles, and 18x contention reduction via thread-local packet pools at 32 threads. These figures measure the software scheduling path, not end-to-end network latency: Net does not claim nanosecond physical networking, only that the software bus need no longer be the dominant bottleneck. The core runtime compiles to a 2.6MB library—4.06MB with the full folded-state stack (durable logs, cross-model query, blob storage, supervisor)—and is covered by over 4,600 tests.

Keywords: mesh networking, non-localized event bus, edge computing, causal ordering, zero-copy forwarding, capability-based routing, encrypted transport, partition healing

—

> Reading note. This is a public working draft. All benchmarks measure *software packet-scheduling overhead* on commodity hardware—not end-to-end wire latency, NIC latency, or physical propagation. Net does not claim nanosecond physical networking; it claims that the software coordination path can be made thin enough that NIC, topology, and physics become the dominant remaining costs. Real multi-hop network evaluation, formal verification of partition reconciliation, and hardware-backed attestation remain future work.

mesh networking | non-localized event bus | edge computing | causal ordering | zero-copy forwarding | capability-based routing | encrypted transport | partition healing

Introduction

The dominant networking paradigm—TCP/IP with best-effort delivery—was designed for an era of scarcity. ARPANET assumed that nodes, bandwidth, and routes were precious, and that the network must guarantee delivery even through partial destruction (1). The resulting protocol stack prioritizes reliability over latency, using queues at every hop to absorb bursts and cooperative congestion control to prevent collapse.

This design is the wrong default for systems where data is abundant, nodes are abundant, and the external pressure—sensor streams, token generation, market feeds—is continuous and unyielding. In such environments, a delivery guarantee becomes a liability: promising to deliver data that will overwhelm the receiver. The bottleneck is not delivery but processing. A guarantee that data arrives at a buffer does not guarantee the receiver can act on it in time.

Real-time networks (CAN bus, EtherCAT, TSN, military MANETs) solve the latency problem by controlling the physical layer: fixed topologies, dedicated hardware, time-slotted access. They achieve deterministic timing because they own the wire. They do not scale to heterogeneous, dynamic, or adversarial environments.

Net occupies the gap between these paradigms. It achieves latency-first scheduling on commodity hardware over commodity networks—we use *latency-first* in the systems sense of avoiding unbounded queues and coordination stalls, not in the hard-real-time sense of guaranteed deadlines over arbitrary networks. It does not guarantee timing through infrastructure control but through architectural choices: drop instead of queue, route around instead of wait, observe instead of coordinate, derive instead of query. The individual techniques—event sourcing (2), process migration (3), causal ordering (4), capability scheduling (5), self-healing mesh (6)—are established. To our knowledge, no prior system has composed them into a single latency-first runtime with a non-localized event-bus abstraction at nanosecond scheduling overhead, because no prior transport layer was fast enough to make the composition viable. You cannot migrate small state in microseconds if the coordination path adds milliseconds. You cannot make sub-microsecond recovery decisions if the failure-handling path depends on heavyweight connection-oriented control flow.

Net's central abstraction is a non-localized event bus. In brokered systems such as Kafka the bus has a location—a broker cluster, a partition leader, a data center; in single-process systems such as the LMAX Disruptor it is a local ring buffer. In Net the logical bus is the mesh itself. This paper evaluates Net's software scheduling path, not physical network propagation: the results show that parsing, routing, forwarding, authorization, and failure-recovery decisions execute at nanosecond to sub-microsecond scale on commodity hardware. End-to-end latency still depends on NICs, interconnects, topology, payload encryption, and the speed of light. The claim is narrow and deliberate—for latency-first coordination workloads, the software bus need no longer be the dominant bottleneck.

This paper makes the following contributions:

1. A **layered mesh architecture** (Section 3) that separates transport, identity, authorization, hierarchy, state, compute, extensibility, and resilience into composable layers, each operating at microsecond or sub-microsecond overhead.

2. A **68-byte header** (Section 4) laid out for natural 8-byte-aligned reads, with its routing fast-path in the first cache line, enabling zero-copy multi-hop forwarding where intermediate nodes make routing decisions from a single cache-line read without decrypting payloads.

3. A **causal state model** (Section 6) using 32-byte causal links with compressed horizons, providing per-entity ordering without global consensus, and an honest discontinuity mechanism that forks entities with documented lineage rather than silently recovering.

4. A **bloom-filter authorization scheme** (Section 5.2) delivering ~20ns per-packet access control for named hierarchical channels.

5. **Comprehensive micro-benchmarks** (Section 9) on two architectures (ARM and x86) demonstrating that the software layer is no longer the bottleneck for packet scheduling.

—

2. Design Principles

Net is organized around six principles that collectively invert the assumptions of best-effort networking.

2.1 Self-Preservation as the Only Axiom. A node must survive by not getting overloaded. Everything else follows. Nodes reject work they cannot process within a time window. Dropping stale work or selecting an alternate path can be a nanosecond-scale local scheduling decision. Waiting for a congested node's guaranteed response costs milliseconds. When dropping is cheaper than waiting, delivery guarantees become overhead.

2.2 State Propagation, Not Connection Maintenance. Traditional networking treats the connection as the primary object. Net propagates state. Connections are ephemeral transport—the current shortest path between where state is and where it needs to be. When a path breaks, state moves. Identity lives in the cryptographic chain, not in the socket. A node can change IP addresses, traverse NAT, switch interfaces, and remain the same entity.

2.3 Observational Consistency. There is no global truth, only local views. Each node observes its neighborhood and derives the state of the wider mesh from those observations. Two nodes may disagree about mesh state at a given instant. Their views are causally consistent within their own observation window. No global consensus, no coordinator, no privileged node.

2.4 Trust Derived from Behavior. Net makes no assumptions about actor goodwill. The protocol is not cooperative in the TCP sense—it does not assume both sides want the connection to succeed. Trust is derived from observation: a node that claims capacity it does not have is routed around when its silence or latency betrays it.

2.5 The Non-Localized Event Bus. Conventional event buses—LMAX Disruptor (7), kernel ring buffers, and distributed brokers like Kafka (2)—are localized. The Disruptor is a single-process ring buffer. Kafka is a cluster of brokers at fixed network addresses. In both cases, the bus has a location: a process, a machine, a data center. Producers and consumers are aware of this location and connect to it.

Net's event bus has no location. The sharded ring buffers on each node are local speed buffers, but the logical event bus spans the mesh. A producer on node A and a consumer on node C interact through the same abstraction regardless of whether they are on the same machine, the same subnet, or separated by five relay hops. The mesh handles routing, encryption, forwarding, and failure recovery transparently. The bus is not *at* a location—it *is* the mesh.

```
Brokered (Kafka, Pulsar):  Producer --> [ broker ] --> Consumer    bus = fixed
   address
Local (LMAX Disruptor):    Producer --> [ ring buf ] --> Consumer  bus = one process
Non-localized (Net):       Producer <==>  ( mesh )  <==> Consumer   bus = the nodes
```

In brokered and single-process systems the bus has an address that producers and consumers must reach. In Net the bus is the mesh itself: any participating node carries it, relay nodes forward encrypted bytes they cannot read, and adding or losing a node changes capacity, not availability.

This distinction has three consequences. First, there is no broker to provision, scale, or fail over. The bus exists wherever participating nodes exist. Adding a node adds capacity. Removing a node triggers rerouting, not an outage. Second, the bus is encrypted end-to-end: relay nodes forward events they cannot read, unlike broker-based systems where the broker holds plaintext. Third, the bus is causally ordered per-entity, not globally ordered per-partition. This eliminates the partition-leader bottleneck that limits Kafka's throughput and creates the hotspot problem in distributed brokers.

The non-localized bus is what makes processing without storage viable. Events exist in the ring buffers of the nodes they are passing through, for as long as they are relevant, and then they are gone. No broker owns the data as a durable coordination point; it is in transit, resident only in the buffers of nodes currently carrying or processing it unless an explicit persistence layer stores it. Any node with matching capabilities can process it. If that node dies, another picks it up. Storage becomes a choice (via persistence adapters), not an architectural requirement. Durability, when needed, is layered explicitly—through append-only causal logs, content-addressed artifact stores, or application-defined persistence adapters—rather than forced through a broker-backed storage model on the hot path. This separates the latency-critical event path from durability decisions instead of coupling them.

A fourth consequence is the programming model: event consumption is location-transparent. A `MeshDaemon` calls `process(event)` and receives `CausalEvent`s. The daemon does not know—and cannot determine from the API—whether the event originated on the same node, a neighbor one hop away, or a node five hops and two subnet boundaries away. The interface resembles a local function call: synchronous, taking a reference, returning output payloads. The mesh resolved the routing, decrypted the payload, validated the causal chain, and delivered the event before the daemon saw it. From the daemon's perspective, every event is local. This is analogous to location-transparent actor messaging in Erlang/OTP (3), but operating on causally-ordered event streams rather than point-to-point messages, and at nanosecond scheduling overhead rather than microsecond message-passing. The result is that code written for a single-node prototype runs unmodified on a multi-hop mesh. The deployment topology is a runtime decision, not a code change.

2.6 Queues as Failure, Not Virtue. In best-effort networks, queues absorb bursts. In Net, every nanosecond a packet sits in a queue is latency added. A queue means a node accepted work it could not immediately process—a violation of the self-preservation axiom. Ring buffers have fixed capacity. When full, old data is evicted or new data is dropped. There is no unbounded growth.

This creates a fundamental incompatibility with best-effort systems at the queue level. A Net node operating at 10M+ events/sec will fill a TCP socket buffer before a single context switch occurs on the receiving end. TCP's backpressure signals (window scaling, congestion control) operate orders of magnitude too slowly. This is why Net runs its own transport over UDP rather than layering on TCP: the two models are physically incompatible.

Dropping is not indiscriminate. Net distinguishes reliability by channel and workload (Section 4.6): for freshness-dominant streams—telemetry, perception, market ticks—discarding superseded data is correct, because a stale value is wrong input, not late truth. For durable artifacts, causal logs, or state that must be preserved, the same transport offers reliable modes and explicit persistence layers. The inversion is not "drop everything"; it is "never let stale or unprocessable work accumulate unboundedly in the hot path."

—

3. Architecture

Net is organized into nine layers, each providing a distinct concern. All layers share a common 68-byte header format and operate within the same process. The layers are:

A semantic **behavior plane** spans layers 1–3, providing capability announcement and indexing, API schema discovery, device autonomy rules, distributed context propagation (tracing), load balancing, proximity-aware routing, and safety envelope enforcement.

—

Layer	Concern	Key Mechanism
0. Transport	Encrypted UDP, packet pools, forwarding	ChaCha20-Poly1305, Noise NK, zero-alloc pools
1. Identity	Cryptographic entity binding	ed25519 keypairs, BLAKE2s-derived origin hashes
2. Channels	Named authorization endpoints	Bloom-filter AuthGuard, capability filters
3. Subnets	Hierarchical topology	4-level 32-bit encoding, gateway enforcement
4. State	Causal distributed ordering	32-byte CausalLinks, compressed horizons
5. Compute	Migratable event processors	MeshDaemon trait, 6-phase migration
6. Subprotocols	Extensibility	Registry, version negotiation, opaque forwarding
7. Continuity	Observational integrity	Continuity proofs, causal cones, fork records
8. Contested	Partition resilience	Correlated failure detection, log reconciliation

4. Transport Layer

The transport and runtime implementation was internally named the Blackstream Layered Transport Protocol; *Net* (Network Event Transport) is the public name used throughout this paper.

4.1 Wire Format. Every Net packet begins with a 68-byte header, laid out so its `u64` fields sit on natural 8-byte boundaries for single-instruction reads. The fields a forwarding node needs—magic, version, flags, priority, hop limits, subprotocol and channel hashes, session/stream/sequence, origin hash, and subnet—all fall within the first 64 bytes; only `payload_len` and `event_count`, which a relay never inspects to forward, occupy the four bytes past the first cache line. A forwarding node therefore still makes its routing decision from a single cache-line read, decrements TTL, increments hop count, and forwards without decrypting the payload.

```
Offset  Field              Size    Used by
 0      MAGIC (0x4E45)      2B     All (validation)
 2      VERSION             1B     All (compatibility)
 3      FLAGS               1B     Transport (reliability, handshake)
 4      PRIORITY            1B     Router (scheduling)
 5      HOP_TTL             1B     Proxy (forwarding limit)
 6      HOP_COUNT           1B     Proxy (loop detection)
 7      FRAG_FLAGS          1B     Transport (fragmentation)
 8      SUBPROTOCOL_ID      2B     Subprotocol registry
10      CHANNEL_HASH        2B     Channel authorization
12      NONCE              12B     Crypto (AEAD)
24      SESSION_ID          8B     Session management
32      STREAM_ID           8B     Stream multiplexing
40      SEQUENCE            8B     Reliability, ordering
48      ORIGIN_HASH         8B     Identity binding
56      SUBNET_ID           4B     Subnet gateway
60      FRAGMENT_ID         2B     Fragmentation
62      FRAGMENT_OFFSET     2B     Fragmentation
64      PAYLOAD_LEN         2B     Parsing
66      EVENT_COUNT         2B     Batch processing
```

Every field is read by at least one layer. The maximum packet size is 8,192 bytes; the maximum payload is 8,108 bytes (packet minus 68-byte header minus 16-byte Poly1305 tag).

4.2 Encryption. Key exchange uses the Noise NKpsk0 pattern (8): the initiator is anonymous, the responder's static key is known in advance, and a pre-shared key provides symmetric authentication. Payload encryption uses ChaCha20-Poly1305 AEAD (9) (implemented via the `ring` cryptography library) with counter-based nonces, eliminating nonce-reuse risk without requiring randomness on the hot path. Each session derives separate TX/RX keys from the Noise handshake. Headers are never encrypted—only payloads.

4.3 Zero-Allocation Packet Pools. Pre-allocated `PacketPool` instances provide reusable byte buffers. `ThreadLocalPool` eliminates contention entirely: each thread has its own pool with no shared state. At 32 threads, thread-local pools achieve an 18x throughput advantage over shared pools (70.4M vs. 3.89M acquire/release per second on M1 Max; 120.9M vs. 6.45M on i9-14900K).

4.4 Adaptive Batching. An `AdaptiveBatcher` dynamically sizes packet batches based on observed latency (target: 100us) and queue depth. When burst is detected (queue depth > 100), batch sizes increase. An exponential moving average of batch

latency smooths adaptation. The batcher's `optimal_size()` decision is sub-nanosecond (~0.8–1ns); a full record-and-resize cycle is 4.4ns (M1) to 8.1ns (i9).

4.5 Multi-Hop Forwarding. The `NetProxy` forwards packets without decrypting payloads. It reads the 68-byte header, decrements TTL, increments hop count, and forwards in a single operation. Per-hop latency scales linearly: 61.7ns for 1 hop, 271.1ns for 5 hops (M1 Max; 53.4ns to 189.5ns on i9-14900K)—roughly 52ns/hop on M1 and 34ns/hop on i9. The forwarding path allocates nothing.

4.6 Reliability Modes. Two modes implement a common `ReliabilityMode` trait:

- **FireAndForget**: Zero overhead. No per-packet tracking. Suitable for sensor telemetry where the latest value supersedes the previous.
- **ReliableStream**: Per-stream sequence tracking with selective NACKs. The receiver identifies missing sequence numbers; the sender retransmits only the gaps. Timeout-driven retransmission handles lost NACKs.

4.7 Failure Detection. A heartbeat-based `FailureDetector` tracks node health with configurable timeout and miss thresholds. `NodeStatus` progresses through Healthy, Suspected, and Failed states. A `CircuitBreaker` prevents cascading failures by temporarily blocking traffic to failing nodes (check overhead: ~10ns). A `RecoveryManager` evaluates alternate routes; a full fail-and-recover cycle completes in 291ns (M1 Max) and 280ns (i9-14900K).

4.8 Swarm Discovery. `Pingwave` is a 24-byte neighbor discovery protocol that floods the mesh with TTL-bounded propagation. Nodes emit periodic heartbeats; receivers construct a `LocalGraph` representing their k-hop neighborhood. `CapabilityAd` messages announce hardware capabilities (GPU vendor/model, accelerators, memory, CPU cores) and software capabilities (tools, models, modalities, tags). Capability announcements fold into a single queryable index (Section 9.5): a `has_gpu` check on a populated set runs in ~40ns and a single-tag filter in ~57ns on M1 Max, while a selective tag query against the fold stays flat at ~2us regardless of mesh size.

—

5. Identity and Authorization

5.1 Cryptographic Identity (Layer 1). Every entity is identified by a 32-byte ed25519 public key. All other identifiers are deterministically derived:

- **origin_hash** (8 bytes): BLAKE2s-MAC of the public key keyed with `"net-origin-v1"`, truncated to its low 8 bytes. Written into every outgoing packet header.
- **node_id** (8 bytes): BLAKE2s-MAC keyed with `"net-node-id-v1"`, truncated. Used in swarm routing.

Domain-separated key derivation prevents cross-domain collisions. An `OriginStamp` caches both derived values at session creation; per-packet overhead is a single u64 field write (~1ns).

Permission tokens are 159-byte ed25519-signed structures authorizing a subject entity to perform specific actions (publish, subscribe, admin, delegate) on specific channels. Tokens carry expiry timestamps, delegation depth limits, and unique nonces for revocation. Delegation restricts scope to the intersection of the parent's permissions with decremented depth. A `TokenCache` backed by `DashMap` provides sub-microsecond per-channel lookup. Token verification occurs at subscription time, not per-packet.

5.2 Channels and Bloom-Filter Authorization (Layer 2). Channels are hierarchical named endpoints (e.g., `sensors/lidar/front`). A `ChannelName` is validated against format rules (max 255 bytes, alphanumeric plus `-_./`) and hashed to a `u16` via xxh3 for wire-speed filtering. `ChannelConfig` attaches policy to each channel: visibility scope, capability filters for publish/subscribe, token requirements, priority, reliability mode, and rate limits.

The `AuthGuard` combines a bloom filter with a verified-positive cache for O(1) per-packet authorization. The bloom filter occupies 2^15 bits (4KB), fitting entirely in L1 cache. The fast path:

1. Compute a bloom key from `(origin_hash, channel_hash)` via xxh3 (~1ns).
2. Probe two bloom-filter positions via atomic reads (no locks).
3. If either bit is zero, return `Denied` (no false negatives).

4. Probe the verified `DashMap` cache; return `Allowed` on hit, `NeedsFullCheck` on miss.

Total fast-path latency: ~20ns for the per-publish authorization check. Authorization pairs are inserted at subscription time (slow path). Revocation removes from the verified cache; the bloom filter's false positives now trigger full checks that fail.

Scope of enforcement. Net authenticates, routes, and filters; it does not replace application-level policy. The transport carries identity, capability, and authorization envelopes, but resource owners—daemons, tools, devices, or local agents—remain responsible for enforcing the policies attached to the *consequences* of an action. Net decides whether a request is permitted to arrive; the endpoint that performs the work decides whether to honor it.

—

6. Causal Distributed State (Layer 4)

6.1 Causal Links. Every event produced by an entity carries a 32-byte `CausalLink`:

```
origin_hash:      8 bytes (u64)  -- entity identity
horizon_encoded:  8 bytes (u64)  -- compressed observed horizon (64-bit bloom)
sequence:         8 bytes (u64)  -- monotonic per-entity
parent_hash:      8 bytes (u64)  -- xxh3(prev_link || prev_payload)
```

All four fields are 64-bit, so the 32-byte layout carries no padding. The `parent_hash` chains each event to its predecessor, providing structural integrity. Tamper resistance is provided by the transport layer's AEAD encryption.

A `CausalChainBuilder` maintains per-entity chain state and produces new links. Chain validation verifies that each event's `parent_hash` matches the hash computed from the previous event's link and payload.

6.2 Compressed Horizons. Each entity maintains an `ObservedHorizon`: a map from `origin_hash` to the latest observed sequence number from that entity. For wire transmission, the `HorizonEncoder` compresses this into an 8-byte (64-bit) bloom sketch using xxh3. Remote observers can perform approximate causal queries (false positives possible, false negatives impossible); local nodes with the full horizon get exact answers.

6.3 Entity Logs. `EntityLog` is an append-only log of `CausalEvent`s for a single entity, with chain validation enforced on every append. State snapshots (`StateSnapshot`) capture entity state at a point in time—daemon state, chain head, and horizon—for migration and recovery.

—

7. Compute Runtime (Layer 5)

7.1 MeshDaemon Trait. The `MeshDaemon` trait defines stateful or stateless event processors:

```
trait MeshDaemon: Send + Sync {
    fn name(&self) -> &str;
    fn requirements(&self) -> CapabilityFilter;
    fn process(&mut self, event: &CausalEvent) -> Result<Vec<Bytes>, DaemonError>;
    fn snapshot(&self) -> Option<Bytes>;
    fn restore(&mut self, state: Bytes) -> Result<(), DaemonError>;
}
```

All methods are synchronous for WASM compatibility. The runtime automatically wraps output payloads in `CausalLink`s. A `Scheduler` places daemons on nodes whose capabilities match the daemon's requirements.

7.2 Six-Phase Migration. Daemon migration preserves causal chain continuity via a strict state machine:

1. **Snapshot**: Serialize daemon state on the source node.
2. **Transfer**: Send snapshot to the target via subprotocol 0x0500.
3. **Restore**: Call `restore()` on the target; begin buffering incoming events.
4. **Replay**: Replay buffered events on the target.
5. **Cutover**: Atomic routing switch—new events go to the target.

6. **Complete**: Clean up the source.

Phase transitions are validated; calling a transition method in the wrong phase returns an error. The snapshot's origin hash is verified against the daemon being migrated. Events arriving during migration are buffered and replayed after restore, ensuring zero event loss.

—

8. Observational Continuity and Partition Healing

8.1 Continuity Proofs (Layer 7). A 40-byte `ContinuityProof` demonstrates that an entity's chain is intact over a sequence range without transferring the full log. The proof contains `origin_hash`, `from_seq`, `to_seq`, and the computed `parent_hash` values at both endpoints. A verifier with the entity's log recomputes the hashes and compares.

`CausalCone` answers causal precedence queries: given event E, which other entities' events could have causally preceded E? Local nodes with full horizons get exact answers (`Definite`/`No`); remote observers with only the 8-byte compressed horizon get approximate answers (`Possible`/`No`).

8.2 Honest Discontinuity. When a chain breaks (node crash, data corruption, conflicting chains from the same origin), the system does not silently recover. Instead, `fork_entity()` creates a new entity with a new keypair and a `ForkRecord` documenting the lineage: the original entity's last verified link, the reason for discontinuity, and the new entity's identity. The fork genesis has a deterministic sentinel `parent_hash` so any node can verify the fork's legitimacy.

This design makes chain breaks visible. The mesh knows the original entity is discontinued and a fork has taken its place. Events are not lost—the losing chain is preserved as a fork with documented lineage.

8.3 Correlated Failure Detection (Layer 8). A `CorrelatedFailureDetector` wraps the base `FailureDetector` with a time-windowed correlation layer. When failures within a configurable window (default: 2s) exceed a threshold fraction of tracked nodes (default: 30%), the detector classifies the cause:

1. Collect `SubnetId`s of failed nodes.
2. Walk up the hierarchy via `parent()`, counting failures per ancestor.
3. If any ancestor accounts for >= 80% of failures, classify as `SubnetFailure` (likely partition).
4. Otherwise, classify as `BroadOutage` (likely infrastructure).

During mass failure, recovery is throttled to `max_concurrent_migrations` (default: 3) to prevent recovery storms.

8.4 Partition Healing and Log Reconciliation. When nodes from the "other side" of a partition reappear in `FailureDetector` recovery events, the `PartitionDetector` transitions through Suspected -> Confirmed -> Healing -> Healed phases. After healing, `reconcile_entity()` merges divergent `EntityLog`s:

1. Both sides empty after split: `AlreadyConverged`.
2. One side empty: `Catchup` (replay missing events).
3. Both sides have events and chains are identical: `AlreadyConverged`.
4. Chains diverge at some sequence: **longest chain wins**. Equal length: **lower `parent_hash` wins** (deterministic tiebreak). Losing chain becomes a `ForkRecord`.

Both sides reach the same conclusion independently. No coordination protocol is needed.

—

9. Evaluation

All benchmarks measure **packet scheduling**: the time to process, route, encrypt, and queue a packet for transmission. They do not include NIC transfer, wire latency, or speed-of-light propagation. The software layer is what these benchmarks demonstrate is no longer the bottleneck.

Test systems: Apple M1 Max (macOS), Intel i9-14900K at 5GHz (Windows 11). Core-crate numbers captured 2026-06-12; SDK ingestion 2026-06-15.

Operation	M1 Max	i9-14900K
Header serialize	2.19ns (456M/s)	1.21ns (829M/s)
Header deserialize	2.35ns (426M/s)	1.61ns (622M/s)
Routing header serialize	0.62ns (1.60G/s)	0.51ns (1.97G/s)
Routing header forward	0.57ns (1.75G/s)	0.20ns (4.96G/s)
Routing lookup (hit)	37.7ns (26.5M/s)	38.1ns (26.2M/s)
Decision pipeline	37.5ns (26.7M/s)	38.5ns (26.0M/s)

Hops	M1 Max	i9-14900K
1	61.7ns (16.2M/s)	53.4ns (18.7M/s)
2	116.5ns (8.59M/s)	88.6ns (11.3M/s)
3	160.0ns (6.25M/s)	121.1ns (8.26M/s)
5	271.1ns (3.69M/s)	189.5ns (5.28M/s)

9.1 Header and Routing Operations. Sub-nanosecond routing header operations demonstrate that the software overhead for forwarding decisions is dominated by the DashMap probe for routing table lookup, not by serialization or parsing.

9.2 Multi-Hop Forwarding. Forwarding latency scales linearly with hop count (~52ns/hop on M1, ~34ns/hop on i9). No amplification effects. At 5 hops, the total scheduling overhead remains under 300ns—well within the latency budget for edge-to-edge coordination on a campus network where the physics floor is 33us.

Concurrent forwarding scales with threads: 16 threads achieve 9.20M/s (M1) and 10.5M/s (i9).

Payload	M1 Max	i9-14900K
64B	301ns (203 MiB/s)	213ns (287 MiB/s)
256B	477ns (512 MiB/s)	278ns (878 MiB/s)
1KB	908ns (1.05 GiB/s)	541ns (1.76 GiB/s)
4KB	2.89us (1.32 GiB/s)	1.53us (2.50 GiB/s)

9.3 Encryption. The i9-14900K's wider SIMD throughput provides a significant advantage at larger payload sizes. The M1 Max is competitive at smaller payloads. End-to-end packet build (50 events, including serialization, framing, and encryption) completes in 2.43us (M1) and 1.43us (i9). These AEAD costs fall on endpoints: relay nodes forward on the routing header alone and never decrypt payloads, so the per-hop forwarding figures (Section 9.2) and the encryption figures here are independent—intermediate nodes pay the former, never the latter.

Operation	M1 Max	i9-14900K
Heartbeat (existing)	39.8ns (25.2M/s)	69.3ns (14.4M/s)
Status check	15.1ns (66.2M/s)	15.2ns (65.9M/s)
Circuit breaker	9.55ns (104.7M/s)	11.1ns (90.0M/s)
Recovery evaluate	257.5ns (3.88M/s)	354.9ns (2.82M/s)
Full fail+recover	290.8ns (3.44M/s)	279.6ns (3.58M/s)

9.4 Failure Detection and Recovery. A complete failure detection and recovery cycle—marking a node failed, evaluating alternates, selecting a recovery target, and updating routing—completes in under 300ns. The `check_all` health sweep is a periodic O(n) maintenance scan (run once per heartbeat interval, not per packet) that scales linearly: 5,000 nodes scanned in 54.9us (M1) and 87.5us (i9), at 57–91M checks/sec.

9.5 Capability System. Capability announcements fold into a single queryable index. The per-match checks below run against a fully populated set:

Operation	M1 Max	i9-14900K
`has_gpu` (populated set)	40.5ns (24.7M/s)	40.6ns (24.6M/s)
Single tag filter	57.1ns (17.5M/s)	59.7ns (16.7M/s)
Require GPU filter	46.7ns (21.4M/s)	42.8ns (23.4M/s)
Announcement to fold (create)	3.41us (293K/s)	2.75us (364K/s)
Fold insert (per node)	40.3us (24.8K/s)	31.9us (31.4K/s)

Querying the fold by mesh size shows the key property: a *selective* tag (few matches) stays flat regardless of mesh size, while a *broad* tag (many matches) scans its match set and grows with it.

Nodes	Broad tag (M1 / i9)	Selective tag (M1 / i9)
1,000	9.50us / 7.97us	1.90us / 1.52us
10,000	108us / 172us	1.90us / 2.85us
50,000	644us / 1.21ms	1.91us / 2.94us

The flat selective-query cost (~2us through 50,000 nodes in this benchmark) is the important scaling property: placement queries naming a specific capability do not slow down with total mesh size, only with match-set size.

Threads	Shared Pool	Thread-Local Pool	Advantage
8	4.60M/s	67.8M/s	14.7x
16	4.23M/s	72.7M/s	17.2x
32	3.89M/s	70.4M/s	18.1x

9.6 Thread-Local Pool Scaling. *(M1 Max, 10,000 acquire/release per thread)*

Thread-local pools eliminate contention entirely. The shared pool's throughput degrades slightly with thread count due to CAS contention on the lock-free queue; the thread-local pool scales monotonically.

SDK	Method	Throughput	Latency
Go	raw (9B)	8.69M/s	115ns
Go	batch (1000)	7.51M/s	133ns/event
Python	raw (9B)	6.88M/s	0.15us
Python	batch (1000)	9.53M/s	0.10us
Node.js	batch	6.07M/s	0.16us
Node.js	single	4.34M/s	0.23us
Bun	batch	6.91M/s	0.14us
Bun	single	4.57M/s	0.22us

9.7 SDK Ingestion. All language bindings wrap the same Rust core via FFI. All exceed 4.3M events/sec on single-event ingestion and 6M+ events/sec on batch. Go leads single-event ingestion at 8.69M/sec with zero allocations on the raw path; Python via PyO3 leads batch at 9.53M/sec. Bun batch ingestion is ~14% faster than Node.js.

—

10. Subnet Hierarchy and Gateway Enforcement (Layer 3)

`SubnetId` packs a 4-level hierarchy (region/fleet/vehicle/subsystem) into a 32-bit integer, with 8 bits per level (256 values each). Parent/child/sibling relationships resolve via bitwise operations. Gateways at subnet boundaries enforce channel visibility by reading only header fields—no decryption, no payload inspection. Four visibility modes control propagation:

- **SubnetLocal**: never crosses boundaries.
- **ParentVisible**: visible to ancestor subnets only.
- **Exported**: forwarded to explicitly listed target subnets.
- **Global**: no restriction (default).

Label-based `SubnetPolicy` assigns nodes to subnets based on capability tags, evaluated in rule order with a configurable default.

—

11. Infinite Extensibility via Subprotocols

Every Net packet carries a `subprotocol_id` field. This is 16 bits in the header — 65,536 possible protocols — and it changes everything about how the mesh evolves.

A vendor builds a custom inference protocol for their hardware. They pick an ID in the vendor range, implement the `MeshDaemon` trait, register it in the `SubprotocolRegistry`, and deploy. The mesh already knows how to route their traffic — the node advertises `subprotocol:0x1000` as a capability tag, the existing `CapabilityIndex` indexes it, and any node that needs that protocol can find a handler through the same query path used for GPU discovery or tool matching. No firmware update. No mesh-wide upgrade. No coordination with anyone.

The critical property is the **opaque forwarding guarantee**: nodes that don't understand a subprotocol forward it anyway. They read the routing header, decide the next hop, and pass the encrypted payload through without inspection. The intermediate node doesn't need to know what's inside. It doesn't need the handler installed. It doesn't even need to know the subprotocol exists. It forwards because the routing header says to, and it can't read the payload even if it wanted to.

This means new protocols deploy incrementally. You upgrade the nodes that need to process the protocol. Every other node in the mesh — every relay, every gateway, every forwarding hop — continues working unchanged. There is no flag day. There is no "upgrade the mesh to support protocol X." The mesh already supports protocol X. It just doesn't know what X means, and it doesn't need to.

Version negotiation happens at session establishment, not per-packet. Peers exchange manifests — compact lists of (protocol ID, version, minimum compatible version) — and compute a `NegotiatedSet` of protocols they both understand. This is a pure function. No coordinator, no registry server, no version authority. Two nodes meet, compare notes, and know what they can talk about.

The consequence is that the mesh is not a fixed protocol. It is a protocol runtime. The transport, encryption, routing, forwarding, failure detection — those are fixed. Everything above them is a subprotocol that can be swapped, extended, versioned, and deployed independently. The mesh doesn't have features. It has a feature space.

—

12. Behavior Plane

A semantic layer spanning the identity, channel, and subnet layers provides:

- **Capability announcements and indexing**: hardware (GPU, accelerators, memory, CPU) and software (tools, models, modalities, tags) capabilities with `DashMap`-backed secondary indexes. Incremental updates via `CapabilityDiff`.
- **Node metadata**: location, network tier, NAT type, topology hints. Queryable by status, region, and custom predicates.
- **API schema registry**: typed endpoint discovery with path, method, parameters, and response schemas. Version-aware announcement and query.
- **Device autonomy rules**: condition-action policies evaluated in priority order against a rule context. Boolean logic over comparison predicates.
- **Context fabric**: distributed tracing with W3C-compatible trace/span IDs, baggage propagation, and configurable sampling.
- **Load balancing**: pluggable strategies (round-robin, least-connections, weighted, random, consistent-hash) with health-aware endpoint selection.
- **Proximity graph**: latency-measured edges augmenting Pingwave discovery, enabling nearest-neighbor queries weighted by both latency and capability match.
- **Safety enforcement**: resource envelopes, rate limits, content policies, kill switches, and audit logging with configurable enforcement modes (enforce/monitor/disabled).

—

13. The Blackwall: Emergent Containment

Net's resilience to uncontrolled propagation is not a single mechanism but an emergent property of every constraint working together:

1. **Backpressure**: nodes go silent when overloaded; no node can be forced to accept more than it can process.
2. **Bounded queues**: ring buffers have fixed capacity; a flood fills a buffer and is evicted.
3. **Fanout limits**: dissemination is controlled by the proximity graph and routing table, preventing O(n^2) explosion.
4. **TTL and propagation limits**: events expire; Pingwaves have a hop radius.
5. **Rate limiting**: per-node, per-peer limits enforced independently via device autonomy rules.
6. **Circuit breakers**: nodes exceeding failure thresholds are temporarily isolated.

Any single mechanism can be overwhelmed. Their composition forms a defense in depth where an event that bypasses one layer encounters the next.

14. Applications

The architecture enables several classes of application that need latency-first coordination rather than brokered request/response. Each illustrates a distinct design principle, not a separate market:

- **AI agent/runtime coordination**: token streams, tool-call results, and guardrail decisions flowing across heterogeneous GPU nodes, with inference routed to capacity via capability matching (Section 9.5) and stateful processors migrated between nodes as load shifts (Section 7). Discovery, placement, and streaming share one substrate; no central broker or control-plane dispatcher sits on the hot path.
- **Distributed sensory networks**: high-rate, time-sensitive streams—wafer-inspection imagery, LIDAR/radar, seismic arrays, precision-agriculture telemetry—where stale data is worse than missing data. Fire-and-forget delivery and bounded ring buffers drop superseded values instead of queuing them, and the proximity graph routes processing to nearby capacity. Delivery is not the bottleneck; timely processing is.
- **Factory robotics**: floor-level coordination among robots, sensors, cameras, and controllers through obstacle-penetrating mesh routing, with sub-microsecond software route updates after failure classification (Section 4.7) and capability-aware compute placement. Local control loops stay on the floor; safety envelopes and rate limits are enforced per node (Section 12).
- **Vehicular sensor mesh**: nearby vehicles exchanging fresh, authorized, latency-scoped perception—a hazard one vehicle can see and another cannot—alongside intent synchronization across a local swarm. Per-vehicle cryptographic identity (Section 5.1), channel authorization, and TTL-bounded validity ensure only fresh, permitted observations influence behavior; stale events expire rather than propagate.
- **Disaster response**: ad-hoc mesh formed from phones, drones, and portable radios with no surviving infrastructure, each device contributing relay, compute, or uplink as available.

Beyond these near-term domains, the same properties—local autonomy, bounded state propagation, tolerance of intermittent connectivity, and no dependence on a stable central broker—extend to remote and orbital systems: satellite and orbital-compute meshes, maritime and aviation networks, and geographically dispersed energy infrastructure, where physical latency and connectivity vary widely but coordination must continue. These are horizon applications rather than first deployments, but they fall within the same design envelope.

—

15. Related Work

Transport protocols. QUIC (10) multiplexes streams over UDP with TLS 1.3 but assumes cooperative endpoints and uses congestion control incompatible with latency-first operation. DPDK (11) and io_uring bypass the kernel for low-latency I/O but do not provide mesh routing, identity, or causal ordering. RDMA achieves microsecond latencies but requires specialized hardware and controlled network environments.

Mesh networking. Military MANETs (OLSR (12), BATMAN (13)) achieve self-healing mesh topology but lack cryptographic identity binding, causal state ordering, and capability-based routing. LibP2P (14) provides peer-to-peer networking primitives but targets internet-scale overlay networks with millisecond latencies rather than nanosecond scheduling.

Event buses. The LMAX Disruptor (7) achieves sub-microsecond event processing via a lock-free ring buffer but is confined to a single process on a single machine. Kafka (2) distributes event streams across brokers but introduces millisecond latencies, requires broker provisioning, and holds plaintext at the broker. Pulsar (15) separates compute from storage but retains the broker model with centralized coordination. All three are localized: the bus has a fixed address that producers and consumers must know. Net's event bus is non-localized—it spans the mesh, has no broker, holds no plaintext at relay nodes, and requires no provisioning. The bus exists wherever participating nodes exist.

Distributed systems. Erlang/OTP (3) provides process migration and supervision trees but over TCP with message-passing overhead. CRDTs (16) provide conflict-free replicated state but typically operate at application-level granularity without the tight integration with transport, identity, and routing that Net provides.

Causal ordering. Vector clocks (4) and Lamport timestamps (17) are foundational. Net's compressed horizons trade exact tracking for an 8-byte wire representation via bloom sketches, providing approximate causal queries at zero per-packet cost. The honest discontinuity mechanism—forking rather than silently recovering from chain breaks—is, to our knowledge, novel in this context.

Capability systems. Kubernetes (5) provides capability-based scheduling but at second-scale granularity over HTTP APIs. Net's capability index operates at nanosecond granularity with inline per-packet filter evaluation.

The positioning across adjacent categories:

System	Primary abstraction	Bus location	Net's difference
LMAX Disruptor	local ring buffer	one process	extends event-bus semantics across nodes
Kafka / Pulsar	brokered log	broker cluster	removes the broker from the hot path; no plaintext at relays
Kubernetes	container scheduling	cluster control plane	schedules capabilities, artifacts, and state across heterogeneous nodes
Serverless	stateless function	cloud platform	models stateful participants, streams, and long-running work
LibP2P	P2P networking toolkit	overlay peers	adds event bus, capabilities, causal state, and compute semantics

—

16. When Net Is Not the Right Tool

Net is not a replacement for TCP, HTTP, Kafka, databases, or serverless functions in ordinary request/response workloads. If a workload is stateless, tolerant of queueing, centrally hosted, and insensitive to freshness or locality, conventional infrastructure is simpler and usually preferable.

Net is intended for systems where participants are long-lived; state and artifacts move between nodes; locality matters; stale data is worse than missing data; central brokers introduce unacceptable latency or authority concentration; capabilities must be discovered dynamically; and nodes may appear, disappear, migrate, or fail. The short form: use serverless for stateless bursts; use Net for stateful presence.

—

17. Limitations and Future Work

No real network evaluation. All benchmarks measure scheduling overhead, not end-to-end network performance. Wire-level evaluation across actual multi-hop topologies is needed.

Partial Layer 8. Contested-environment support covers correlated failure detection and partition healing. Anti-jamming, multi-transport failover, and advanced congestion control require hardware integration and real-traffic development.

No incentive mechanism. The current design assumes a mesh of owned machines. Relay is a cooperative cost of participation. Incentive mechanisms for public, multi-party, or adversarial meshes are out of scope.

Bloom-filter limitations. The AuthGuard bloom filter does not support deletion; revocation relies on verified-cache eviction causing `NeedsFullCheck` failures. Under high churn, the bloom filter's false-positive rate increases until it is rebuilt.

Horizon compression. The 8-byte bloom sketch for horizon encoding provides approximate causal queries. The false-positive rate depends on the number of observed entities. For large meshes, the approximation may become too coarse for precise causal reasoning.

Future work includes wire-level benchmarks over real networks, formal verification of the partition reconciliation algorithm, WASM-based daemon sandboxing, and hardware-accelerated cryptography integration (TPM, SGX, TrustZone) for node attestation.

—

Conclusions

Net demonstrates that composing established distributed systems techniques—event sourcing, process migration, causal ordering, capability scheduling, self-healing mesh—into a single runtime is feasible when the transport layer operates at nanosecond scheduling overhead. The benchmark results are not merely performance metrics; they are existence proofs that the software layer need not be the bottleneck for distributed coordination.

The central abstraction is the non-localized event bus. Conventional event buses have a location: a process (Disruptor), a broker cluster (Kafka), a data center. Net's event bus is the mesh itself. It has no broker to provision, no plaintext at relay nodes, no

partition-leader bottleneck. Events exist in the ring buffers of the nodes they are passing through, for as long as they are relevant. No broker owns the data as a durable coordination point; the data is in transit unless an explicit persistence layer stores it. This is what makes processing without storage viable, and what makes the latency numbers possible: the processing path never touches disk, never queries a broker, never waits on a centralized coordinator.

The 68-byte header, with its routing fast-path in the first cache line, enables zero-copy forwarding. The bloom-filter authorization scheme achieves ~20ns per-packet access control. The causal link model provides per-entity ordering in 32 bytes without global consensus. The honest discontinuity mechanism makes chain breaks visible rather than hiding them. The six-phase migration preserves causal continuity across nodes. The partition reconciliation algorithm achieves deterministic convergence without a coordination protocol.

For a 5km campus, the physics floor is ~33 microseconds; for a factory floor, single-digit microseconds. When coordination is no longer dominated by broker or control-plane latency, closed-loop control across a mesh of autonomous devices, low-latency coordination between robots on a factory floor, and swarm coordination where the mesh reacts faster than any individual node's control loop become feasible. These are what becomes possible when the software coordination path gets out of the way and the dominant remaining constraints are topology, hardware, and physics.

—

ABOUT THIS MANUSCRIPT

This manuscript was prepared using Rχiv-Maker v1.22.0 (18).

EXTENDED AUTHOR INFORMATION

Bibliography

1. Vinton G. Cerf and Robert E. Kahn. A protocol for packet network intercommunication. *IEEE Transactions on Communications*, 22(5):637–648, 1974.
2. Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop*, 2011.
3. Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology, Stockholm, 2003.
4. Colin J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th Australian Computer Science Conference*, pages 56–66, 1988.
5. Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.
6. Charles Perkins, Elizabeth Belding-Royer, and Samir Das. Ad hoc on-demand distance vector (aodv) routing. Technical Report RFC 3561, IETF, 2003.
7. Martin Thompson, Dave Farley, Michael Barker, Patricia Gee, and Andrew Stewart. Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads, 2011.
8. Trevor Perrin. The noise protocol framework. https://noiseprotocol.org/noise.html, 2018.
9. Yoav Nir and Adam Langley. Chacha20 and poly1305 for ietf protocols. Technical Report RFC 8439, IETF, 2018.
10. Jana Iyengar and Martin Thomson. Quic: A udp-based multiplexed and secure transport. Technical Report RFC 9000, IETF, 2021.
11. Data plane development kit (dpdk). https://www.dpdk.org, 2024.
12. Thomas Clausen and Philippe Jacquet. Optimized link state routing protocol (olsr). Technical Report RFC 3626, IETF, 2003.
13. Axel Neumann, Corinna Aichele, Marek Lindner, and Simon Wunderlich. Better approach to mobile ad-hoc networking (b.a.t.m.a.n.), 2008. Internet-Draft, IETF.
14. libp2p: A modular network stack. https://libp2p.io, 2024.
15. Sijie Merli, Matteo Spliet, Guo Sijie, and Boyang Banerjee. Apache pulsar: A distributed messaging and streaming platform. In *Proceedings of ACM SIGMOD*, 2022.
16. Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400, 2011.
17. Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
18. Bruno M. Saraiva, António D. Brito, Guillaume Jaquemet, and Ricardo Henriques. Rxiv-maker: an automated template engine for streamlined scientific publications, 2025. doi:10.48550/arXiv.2508.00836.